



WIFI ENABLED FIRE EXTINGUISHING DRONE



U18ECP8701 – PROJECT PHASE II

A PROJECT REPORT

Submitted by

CHARAN.C. D	19BEC130
MADHUMITHA.K	19BEC160
SRINATH.S	19BEC167

*in partial fulfilment for the award of the degree
of*

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

KUMARAGURU COLLEGE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)

COIMBATORE-641049

ANNA UNIVERSITY: CHENNAI 600025

APRIL,2023

BONAFIDE CERTIFICATE

Certified that this project report titled “**WIFI ENABLED FIRE EXTINGUISHING DRONE**” is the bonafide work of **Mr. CHARAN.C. D [19BEC130], Ms. MADHUMITHA.K [19BEC160], Mr. SRINATH.S [19BEC167]** who carried out the project work under my supervision. Certified further that, to the best of my knowledge the work reported herein does not form part of any other project or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Ms.K. JASMINE, M.E (Ph.D.)

PROJECT SUPERVISOR

Assistant Professor

Department of ECE

Kumaraguru College of Technology

Coimbatore-641049

SIGNATURE

Dr.M. BHARATHI, Ph. D

HEAD OF THE DEPARTMENT

Professor

Department of ECE

Kumaraguru College of Technology

Coimbatore-641049

The candidates with **Register No.: 19BEC130, 19BEC160, 19BEC167** are examined by us in the Project viva-voce held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere thanks to the Management of Kumaraguru College of Technology and **Shri. Shankar Vanavarayar**, Joint Correspondent, Kumaraguru Institutions for the kind support and for providing necessary facilities to carry out the project work.

We would like to express our sincere thanks to our beloved Principal **Dr.D. Saravanan, Ph.D.**, Kumaraguru College of Technology, who encouraged every step of the project.

We would like to thank **Ms. M. Bharathi, Head of the Department, Electronics and Communication Engineering**, for his kind support and for providing necessary facilities to carry out the project work.

We wish to thank with everlasting gratitude to our Project Coordinator **Mr. D. Allin Joe, M.E (Ph.D.) Assistant Professor** Department of Electronics and Communication Engineering for his consistent support throughout the course of this project work.

We are greatly privileged to express our deep sense of gratitude and heartfelt thanks to our Project Guide **Ms.K. Jasmine, M.E (Ph.D.) Assistant Professor** Department of Electronics and Communication Engineering for his expert counselling and guidance to make this project to a great deal of success and, we wish to convey our regards to all teaching and non-teaching staff of ECE Department for their help and cooperation.

Finally, we thank our parents and our family members for giving us the moral support and abundant blessings in all our activities and our dear friends who helped us to endure our difficult times with their unfailing support and warm wishes.

ABSTRACT

One of the most important problems in firefighting is rescue inside a burning building/premises. Firefighting is the act of extinguishing a fire or preventing the spread of fire in order to protect human life, property and the environment. People who work in firefighting are called firefighters. When there is a fire breakout in a building the fire spreads out all over the building area, this makes it difficult for the firefighter to enter and rescue/secure the victims. The methods of fire extinguishing include cooling, starvation and smothering. In a firefighting situation the cooling method is mostly implemented by spraying water over the burning area. This method consumes a lot of time to suppress the flame and it is physically impossible to reach the entire building. The rescue can be done quickly if the fire is suppressed fast. A quicker way of extinguishing the fire can be used like dry-chemical agent. These types of extinguishers are small in size and more effective in extinguishing the fire faster than water. We have proposed a solution that would make the rescues easier for the firefighters. Our solution is that we have designed a drone that carries a dry powder extinguisher to extinguish the fire in the path of firefighter. By looking at the time between both type of extinguishing methods our solution will provide a faster and easier way to carry out the rescue in a burning building and achieve the desired goals.

Keywords: Drone, Fire Extinguisher Ball, Thermal Imaging Camera

LIST OF FIGURES

- Fig.1 Quadcopter
- Fig.2 Pitch vs Throttle
- Fig.3 Body and Inertial Frames
- Fig.4 Fire Extinguisher Ball
- Fig.5 Before Dropping the Extinguisher Ball
- Fig.6 After Dropping the Extinguisher Ball
- Fig.7 Power vs Elevation of the Drone
- Fig.8 Complete design of the drone
- Fig.9 Webpage
- Fig.10 Fire Reducing Status 1
- Fig.11 Fire Reducing Status 2

S.No./ Chapter No.	TITLE OF CONTENTS	PAGE NO
	ABSTRACT	4
	LIST OF FIGURES	5
I	INTRODUCTION 1.1 Objectives	7
II	LITERATURE REVIEW AND RELATED WORK 2.1 Literature Review 2.2 Burchan Aydin, Emre Selvi, Jian Tao and Michael J. Starek, "Use of Fire-Fighting Balls for a Conceptual System of Drone-Assisted Wildfire Fighting" MDPI, March 2019, doi:10.3390/drones3010017. 2.3 Abdulla Al-Kaff, Angel Madridano, Sergio Campos, Fernando Garcia, David Martin and Arturo de la Escalera, "Emergency Support Unmanned Aerial Vehicle Forest Fire Surveillance" MDPI, Sept 2019, doi10.3390/electronics9020260. 2.4 Manuj C., Adarsh M Rao, Rahul S, Suhas C N, Vismay K G, "Design and Development of Semi-Autonomous fire Fighting Drone" Journal of Mechanical and Civil Engineering (IOSR-JMCE) June 2019, e-ISSN: 2394-3343. 2.5 Abinesh D. V, Deepak A. K, Chandraprakash K. Gowtham.M,Ananthi .I," Fire Fighting Drone" IJIERE 2017, Volume 4, e-ISSN: 2394-3343. 2.6 Dr. Ronald T. Wakeham & Dr. John C. Griffith, "Unmanned Aerial Systems in the Fire Service: Concepts and Issues." JMERE, June 2018, eISSN: 2536-80.	10
III	DESIGN AND WORKING OF DRONE 3.1 Working of Quadcopter 3.1.1 Setup 3.1.2 Calibration 3.1.3 Flight Controller 3.2 Working of Fire Extinguisher 3.3 Working of ESP8266 3.4 DHT22 Sensor 3.5 BMP280 Sensor 3.6 Servo Motor 3.7 Fire Extinguishing Ball	15
IV	SIMULATION PARAMETERS AND RESULTS 4.1 Observations 4.2 Results	22
V	CONCLUSION	25
	REFERENCES	26
	APPENDIX I. Code II. Plagiarism Checker III. Paper Submission	27

CHAPTER I

INTRODUCTION

Fires can be devastating, causing harm to people, destruction of property, and financial losses. They can happen unexpectedly, leaving victims with serious injuries and destroyed homes. According to The India Times, in 2017, India was responsible for one-fifth of all fire-related deaths globally, with 27,027 fatalities. This is a startling statistic that highlights the importance of using technology to aid in firefighting efforts.

One such technology is drones. Drones are essentially flying robots that can be remotely controlled or operated through software-controlled flight plans, GPS, and onboard sensors. They have become increasingly popular in recent years for their ability to assist in aerial monitoring, surveillance, and firefighting efforts. The use of drones in firefighting efforts can help protect firefighters from harm and improve efficiency in putting out fires.

Drones can be equipped with thermal cameras and fire extinguisher balls to aid in firefighting efforts. Thermal cameras can detect the presence of fire and locate hotspots even in low-light conditions, where firefighters might have difficulty seeing with their naked eyes. Fire extinguisher balls, when attached to the drone, can strike the flames and activate a dry powder fire extinguisher agent, which can help put out A, B, or C class fires.

In addition to being used in firefighting efforts, drones can be used for post-fire evaluation. After the smoke has cleared, drones can be used to thoroughly evaluate the damage that has been done. This can be especially useful for large-scale fires or natural disasters where it might not be safe for humans to enter the affected area.

The use of drones in firefighting efforts is increasing, and many fire organizations are adopting this technology. In a structural fire, drones can be used to monitor the scene before putting firefighters in danger. This can increase the safety of firefighters and improve efficiency in putting out fires. Drones equipped with spotlights can help firefighters in low-light conditions, making it easier to locate hotspots and potential hazards.

In conclusion, the use of drones in firefighting efforts is becoming increasingly popular. Drones equipped with thermal cameras and fire extinguisher balls can help detect fires and put them out. They can also be used for post-fire evaluation to assess the damage. Drones can improve efficiency and safety in firefighting efforts, protecting firefighters from harm and reducing the risk of further damage. As technology advances, drones will continue to play a vital role in firefighting efforts, helping to protect lives and property.

1.1 OBJECTIVES

Fighting fires has always been a dangerous and challenging task that requires the utmost care and expertise. Traditionally, firefighters have relied on using coolants and other firefighting equipment to extinguish fires from a distance. However, this approach can be time-consuming and puts the lives of firefighters at risk, especially in situations where fires are difficult to reach or contain.

To address these challenges, fire extinguisher drones have been developed as an alternative approach to firefighting. These drones can be operated remotely, and firefighters can use them to drop extinguisher balls filled with chemical powder over targeted areas to extinguish fires more efficiently and without endangering their lives. The drones can reach areas that are difficult for firefighters to access, such as high-rise buildings or enclosed spaces where ground vehicles may not be able to enter.

Furthermore, using fire extinguisher drones provides a defined and controlled firefighting procedure that can help reduce the risk of accidents or unexpected incidents. This method also allows firefighters to monitor and manage the fire more effectively, which can help prevent it from spreading to other areas.

However, despite the potential benefits of using fire extinguisher drones, it is important to note that they are not a replacement for traditional firefighting methods. Instead, they should be used in conjunction with other firefighting equipment and techniques to ensure that fires are extinguished safely and efficiently.

In conclusion, the development of fire extinguisher drones is a significant advancement in the field of firefighting, providing firefighters with a safer and

more efficient method to extinguish fires. As technology continues to advance, it is likely that we will see more innovative solutions to firefighting challenges emerge in the future.

CHAPTER II

LITERATURE REVIEW & RELATED WORK

2.1 LITERATURE REVIEW

A thorough review of all journals related to the chosen topic is one of the most important first steps in starting any research work. These are some examples of the research articles looked at:

2.2 Burchan Aydin, Emre Selvi, Jian Tao and Michael J. Starek, "Use of Fire-Fighting Balls for a Conceptual System of Drone-Assisted Wildfire Fighting" MDPI, March 2019, doi:10.3390/drones3010017.

In 2019, Burchan and colleagues published a study that explored the potential of using drones to fight wildfires. They proposed a system that included fire extinguisher balls as an additional tool for traditional firefighting methods. This system was designed to complement conventional methods rather than replace them. The system comprised six balls, each weighing 0.5 kg, and a hex copter with a payload of 15 kg.

The primary function of the system was to detect spot fires using unmanned aircraft systems (UAS) and assess the likelihood of a wildfire reaching nearby buildings. This information was then transmitted to the firefighting UAS, allowing them to gain situational awareness and coordinate their efforts more effectively. By using drones, the system provided a unique vantage point that was not previously available to firefighters.

However, it was noted that the fire extinguisher balls were not effective against class A and B flames. Class A fires include those that involve ordinary combustibles, such as wood or paper, while class B fires involve flammable liquids and gases. The extinguisher balls were effective against class C fires, which involve electrical equipment, and class D fires, which involve combustible metals. Nonetheless, the system demonstrated promise as a useful supplement to traditional

firefighting methods, especially in difficult-to-reach areas where drones can be deployed quickly and efficiently.

2.3 Abdulla Al-Kaff, Angel Madridano, Sergio Campos, Fernando Garcia, David Martin and Arturo de la Escalera, "Emergency Support Unmanned Aerial Vehicle Forest Fire Surveillance" MDPI, Sept 2019, doi10.3390/electronics9020260.

The passage discusses a technological innovation created by Abdulla Al-Kaff and his team in 2019, which involves the development of unmanned aerial vehicles (UAVs) that can observe forest fires. These UAVs are equipped with internal algorithms that enable them to perform various functions, such as autonomously taking off and landing, planning their trajectories, monitoring fires, and conducting surveillance within a predetermined area.

To detect fires, the UAVs are equipped with temperature sensors and thermal cameras that can detect any sudden changes in temperature and smoke, indicating the presence of a fire. In addition, the UAVs have communication modules that enable them to relay information about the detected fires to the Emergency Response Team (ERT) so that they can take appropriate action.

The use of autonomous UAVs for fire monitoring and surveillance can significantly enhance the effectiveness and efficiency of fire management efforts, particularly in remote or inaccessible areas. By using UAVs, firefighting teams can quickly detect fires and accurately determine their location and intensity, enabling them to take swift and appropriate actions to contain and extinguish them before they spread further. This technology is particularly crucial in preventing catastrophic forest fires that can cause significant damage to the environment and human life.

2.4 Manuj C., Adarsh M Rao, Rahul S, Suhas C N, Vismay K G, "Design and Development of Semi-Autonomous fire Fighting Drone" Journal of Mechanical and Civil Engineering (IOSR-JMCE) June 2019, e-ISSN: 2394-3343.

The passage describes a study conducted by Manuj and colleagues in 2019 that proposes the use of semiautonomous drones for firefighting operations as a safer alternative to risking the lives of firefighters. The study focused on improving the

existing hexacopter to enable it to fly continuously, collect and store GPS data, and perform auto-landing. Additionally, the system was upgraded with firefighting equipment, including a fire extinguisher and other supplies necessary for firefighting.

The study resulted in the development and evaluation of two subsequent models, which produced positive outcomes for the research subject. By using drones, firefighters can safely monitor and respond to fires, particularly in hard-to-reach areas. The drones' ability to fly continuously and gather data allows firefighters to obtain real-time information on the fire's location and intensity, which can help them make more informed decisions and take timely action to mitigate the damage caused by the fire.

Overall, the passage suggests that the use of semiautonomous drones for firefighting operations can be an effective and safe approach to fighting fires while also minimising the risks to firefighters' lives. The study's positive results demonstrate the potential benefits of using technology in emergency response operations.

2.5 Abinesh D. V, Deepak A. K, Chandraprakash K. Gowtham.M, Ananthi .I," Fire Fighting Drone" IJIERE 2017, Volume 4, e-ISSN: 2394-3343.

In 2017, a team of researchers led by Abinesh D. V. developed a quadcopter to analyse burned areas. To achieve their goal, they went through a rigorous process of evaluating various drone designs. After considering different factors such as cost, ease of production, and efficiency, they decided on a quadcopter with an arm length of 220 mm. The researchers chose this design because it was affordable and could be easily produced.

To improve the quadcopter's stability, they decided to use a 10 x 4-inch propeller. However, the team was aware that the drone would be flying in close proximity to fires, which could lead to high temperatures. To mitigate this risk, they carefully selected the components of the propeller, taking into account the high temperatures the drone would encounter when close to a fire.

Overall, the team's careful consideration of different design factors and components resulted in the successful development of a quadcopter capable of analysing burns. This project highlights the importance of carefully evaluating

design choices and considering potential risks when developing innovative technologies.

2.6 Dr. Ronald T. Wakeham & Dr. John C. Griffith, "Unmanned Aerial Systems in the Fire Service: Concepts and Issues." JMER, June 2018, e-ISSN: 2536-8.

In 2018, Dr. Ronald T. and his colleagues published an article discussing the utilisation of unmanned aerial systems (UASs) in the fire department. The article highlighted the importance of certain aspects related to UAS deployment in firefighting operations, such as the airworthiness of drones, command and control systems, and crash prevention measures. Ensuring that the UAS are safe and reliable is crucial, as they are used in critical emergency situations.

To further improve the use of UASs in the fire service, the authors recommended conducting more field testing and collecting feedback from fire chiefs to determine their opinions and concerns regarding the integration of UAS technology into firefighting operations. Collaboration between the Federal Aviation Administration (FAA), fire chiefs, and policymakers is also crucial in drafting laws and regulations governing the use of UAS technology in the fire department.

In summary, the article highlighted the potential benefits of utilising UAS in firefighting operations but also emphasised the importance of ensuring their safety and addressing any concerns related to their deployment. The authors recommended conducting more testing, gathering feedback from relevant stakeholders, and collaborating to develop appropriate regulations for the use of UAS technology in the fire service.

Within this article, we:

- The drone is intended to be configured as an octocopter.
- Only then will the necessary amount of thrust be generated by
- the tiny propellers on the rotors. Each of the following would occur if quadcopter or hex copter configurations were

- The size of the propellers must be increased in order to create the same amount of thrust as an octocopter configuration. Due to the H-shaped design of the main frame, there is adequate room to install the payloads.

CHAPTER III

DESIGN AND WORKING OF DRONE



Fig.1 Quadcopter

3.1 WORKING OF QUADCOPTER

3.1.1 SETUP

- The circuit diagram is used to build the quadcopter.
- The components are connected as per the connections in the circuit diagram.
- The flight controller is connected to the computer using a USB cable.
- Following the upload of the programmes to the controller, further steps can be taken.
- First, the channels of the receiver are assigned a specific value and operation.
 - channel-1=throttle
 - channel-2=yaw
 - channel-3=pitch
 - channel-4=roll

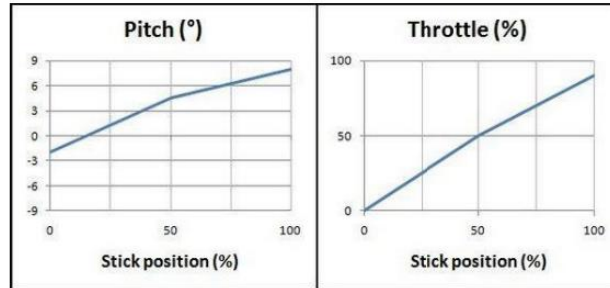


Fig. 2 Pitch vs Throttle

- After channel assignment, the endpoints in the transmitter are measured for real-time operation during flight. Then the gyroscope module is calibrated by tilting.

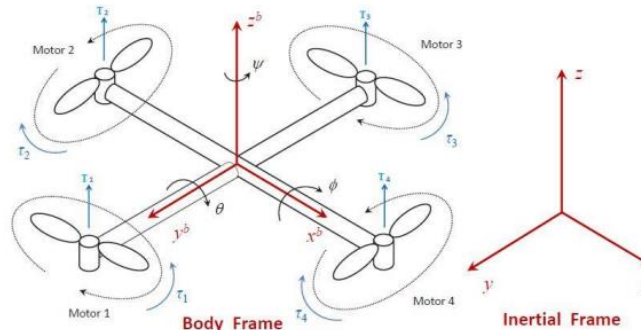


Fig. 3 Body and Inertial Frame

- These values are measured instantaneously for auto-level flight.

3.1.2 CALIBRATION

Calibration is a crucial process in ensuring the proper functioning of electronic devices, especially in the field of radio-controlled devices. One such device that requires calibration is the electronic speed controller, or ESC, which regulates the speed of motors in radio-controlled vehicles, such as drones or RC cars.

The calibration process for an ESC varies among different manufacturers, and it is essential to refer to the manufacturer's manual for specific instructions. Generally, the calibration process involves connecting the power supply and motor to the ESC and waiting for the motor to emit a beep sound.

Once the beep sound is heard, the throttle stick should be moved all the way up, and another beep sound should be heard, which needs to be verified while the throttle is in the minimum position. This process needs to be repeated for all the ESCs that are being used to ensure that they are calibrated correctly.

After the calibration is complete, the second section of the code should be uploaded to confirm the parameters for the radio transmitter, including throttle, yaw, pitch, and roll, and to check the gyroscope values. Any errors found during verification should prompt the user to check the circuit to ensure that the ESCs are functioning correctly.

In conclusion, calibration is a vital step in ensuring the proper functioning of electronic devices, and it is essential to follow the manufacturer's instructions for a particular device. Proper calibration of ESCs is crucial to ensuring the safety and performance of radio-controlled vehicles.

3.1.3 FLIGHT CONTROLLER

- Finally, the flight controller code is to be uploaded. Here the PI D values are to be determined.
- Proportional Control Expression

$$T(s) = \frac{X(s)}{R(s)} = \frac{K_p}{s^2 + 10s + (20 + K_p)}$$

- Proportional Integral Control Expression

$$T(s) = \frac{X(s)}{R(s)} = \frac{K_p s + K_i}{s^3 + 10s^2 + (20 + K_p)s + K_i}$$

- Proportional Derivative Control Expression

$$T(s) = \frac{X(s)}{R(s)} = \frac{K_d s + K_p}{s^2 + (10 + K_d)s + (20 + K_p)}$$

- As this is an auto-level quad, it uses the PID and gyroscope values to level the quad. After uploading the flight controller code, the quad is armed to fly. The

controller changes the corresponding parameters according to the received values from the transmitter. This is how the drone works.

- Drone specifications:

Payload capacity:2kg

Range: 1km

Max Height:100m(approx.)

3.2 WORKING OF FIRE EXTINGUISHER

- The ESP32 cam is set up to take images and stream live over Wi-Fi. A gadget has a connection to the available network through this camera module. The camera's IP address is used to access it from the connected device.
- We can live broadcast and snap pictures using the camera module's website. Additionally, this module has a microSD card storage option.
- Specifications:
 - Communication: Wi-Fi
 - Bandwidth; 2.4ghz
 - Camera: 2MP
 - Storage: Sd card-4gb
- The ESP8266 module is programmed to display and monitor the fire status. It is also capable of dropping the extinguisher. When this circuit is powered on, it is programmed to connect to the specified network.
- It has two programming options.
 - station mode
 - access point mode.
- This module is programmed in station mode for our project.
- You can connect to an accessible network and send data while in this mode.
- The nodemcu module begins showing the webpage at an IP address after configuring the network connection.
- The Wi-Fi network's server is used to access this website. It shows the extinguisher status as well as the temperature, pressure, and fire status. The sensors linked to the nodemcu are continuously read for the parameter values.

- Fire extinguisher balls made of dry powder are the drone's cargo. To put out flames and aid the firefighter, you can drop this ball. By clicking the correct button on the website, the ball can be dropped.
- The servo is programmed to release the extinguisher ball after pressing the button, and the server sends a value to nodemcu. During the procedure, the esp32 cam can be used as a visual aid. It is simple to implement this Wi-Fi communication during any emergency. The circuit for a fire extinguisher function as shown.

3.3 WORKING OF ESP8266

The ESP8266 is a relatively affordable and user-friendly tool for connecting your creations to the internet. The module may function as a station (connect to Wi-Fi) and an access point (create hotspots), allowing it to simply retrieve data and post it to the internet, making the Internet of Things as simple as feasible.

Here, the ESP8266 is utilized to transmit the client's data collection from the drone. In order for us to monitor the sensor data and analyses the current state of the situation—whether a fire is spreading throughout a building or only nearby—the customer will watch these data and take appropriate action. According to the clients' instructions, the drone will release the fire extinguisher ball if the fire is spreading.

3.4 DHT22 SENSOR

In order to establish whether there is a fire, this sensor measures the temperature and calculates the humidity. The DHT22 Sensor is a thermistor that measures air temperature and can monitor temperatures between -30 and 70 degrees.

3.5 BMP280 SENSOR

It is employed to determine height. The altitude of a drone relative to the ground is measured, which is particularly helpful in determining the maximum height at which a drone may fly steadily. The ESP8266 module will send these data to the client.

3.6 SERVO MOTOR

The fire extinguisher ball is dropped by the servo motor. A holder for the fire extinguisher ball is kept below the drone. The customer or user instructs the drone to drop the ball at the precise spot by clicking the drop button on the webpage where all the data is shown.

3.7 FIRE EXTINGUISHING BALL

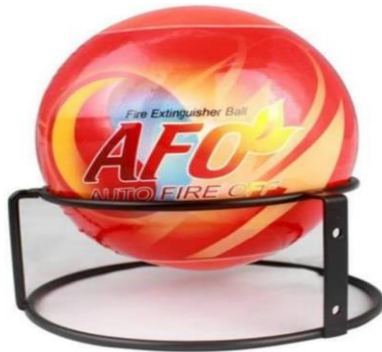


Fig. 4 Fire Extinguisher Ball

A carrier situated below the drone holds the ball. A servo motor controlled by a webpage is used to operate the dropping mechanism. As soon as the ball is dropped into the flame, it responds right away. The dry chemical powder is dispersed over the space quickly by the reaction. The fire's temperature is substantially lowered, allowing the fireman to approach and take additional action. The fire's condition before and after the extinguisher ball was dropped are shown in the graphs below.

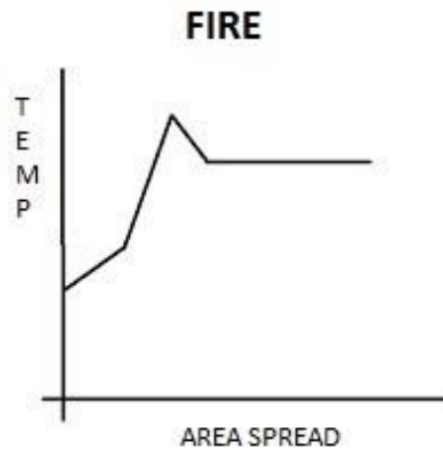


Fig. 5 Before Dropping the Extinguisher Ball

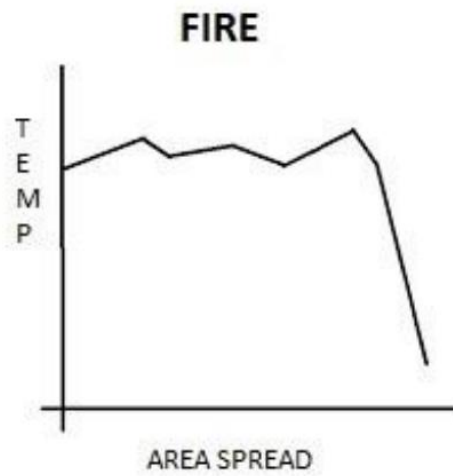


Fig. 6 After Dropping the Extinguisher Ball

CHAPTER IV

OBSERVATIONS AND RESULTS

4.1 OBSERVATIONS

- Max Payload Capacity:1.5kg
- Max Altitude (Without Payload):100m
- Max Altitude (With Payload):50m
- Max Flight Time: 12-15min
- Max Range:50m

4.2 RESULTS

For our drone, we chose a lightweight f450 quadcopter frame. The A212/13T 1000kv BLDC motors are utilized. The drone's propeller of choice is a two-blade (10x4.5) propeller. This offers a 4.5 inch per revolution of thrust force.

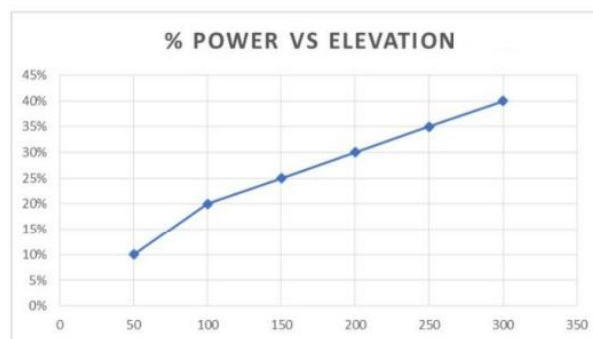


Fig.7 Power vs Elevation of the Drone

The Fire Fighting Drone can enter a burning area and extinguish the fire since it can squeeze through tight places. A fire ball is dropped above the fire using a dropping mechanism after the fire source has been identified. Instead of using water,

fire balls are employed since they can put out fires quickly after being dropped on them.



Fig.8 Complete Design of Drone

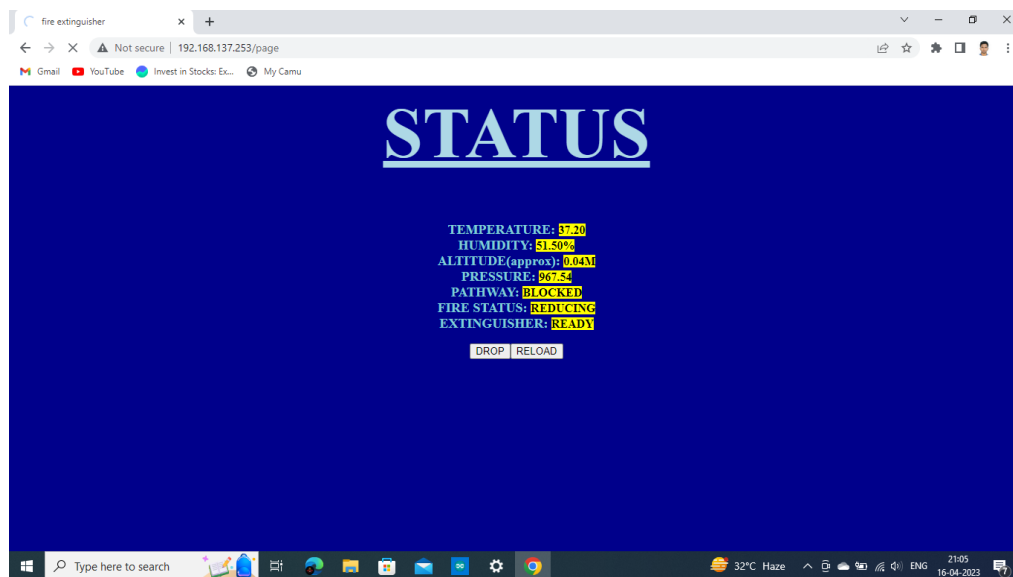


Fig. 9 Webpage

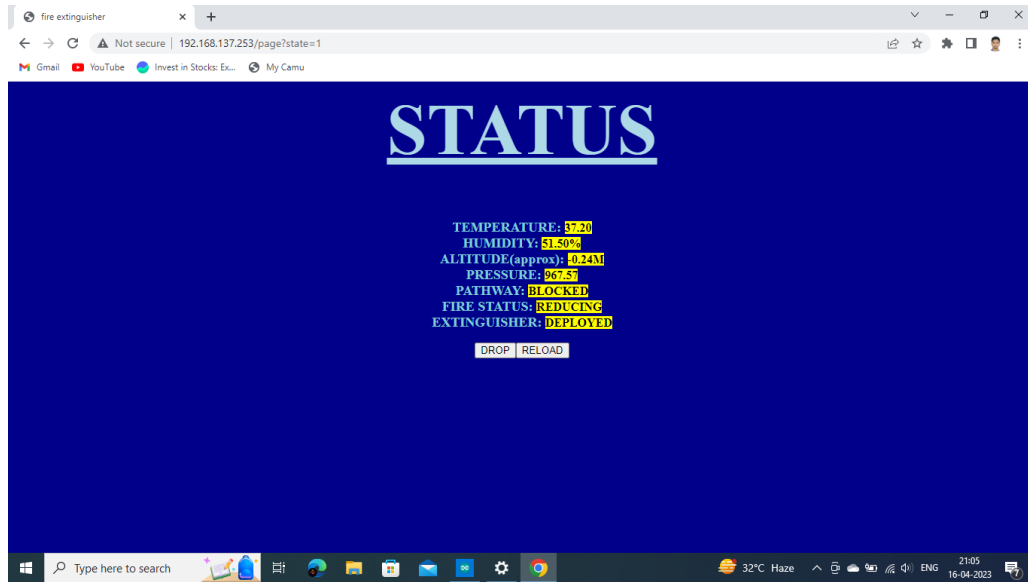


Fig. 10 Fire Reducing Status 1

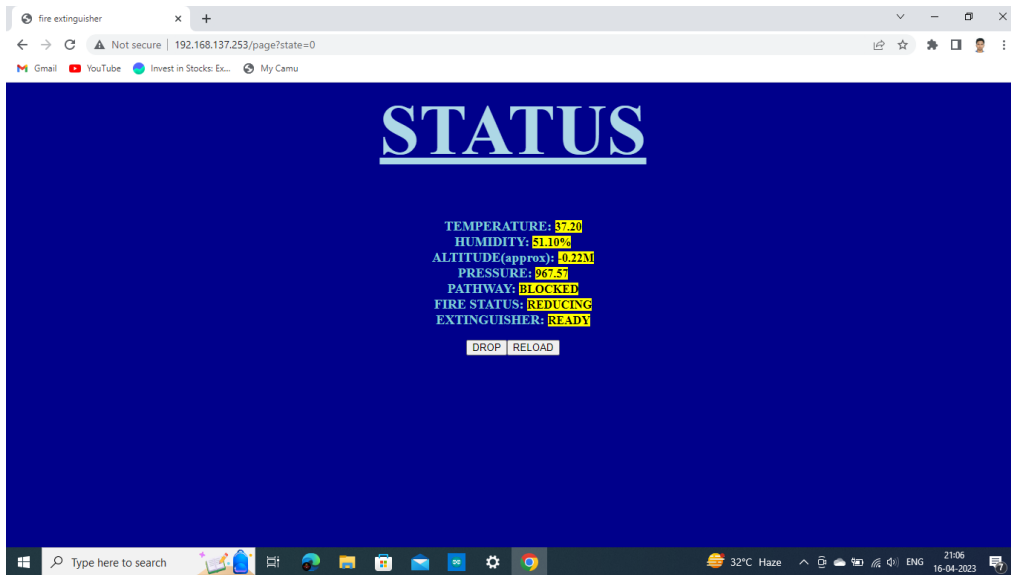


Fig. 11 Fire Reducing Status 2

CHAPTER V

CONCLUSION

Our objective is to develop and deploy a firefighting drone that will aid firefighters in their line of duty during fire incidents. This drone's design enables it to support and transport a 1.5kg payload. A promising 10–15-minute flight time is offered by the drone. This essay presents a method for putting out fires that are out of reach. It demonstrates how a small quadcopter may extinguish the flames when used as directed. This lowers the chance of suffering harm or passing away in a fire accident. Our idea offers real-time information on the local situation by comparing our prototype to the current model. Using the WIFI module on the controller board that was utilized in the circuit, we have used straightforward server-client communication. A camera module linked to the same network as the nodemcu transmits the live visual in addition to the data and status through the webpage. With a basic WIFI network offered by any PDA (personal digital assistant), this mode of communication is effective and simple to adopt in any situation. Our UAV is also inexpensive and easy to use, therefore it may be easily applied in the firefighting division. In conclusion, this WIFI-enabled fire extinguisher drone is a successful use of the firefighting UAVs' current technology.

REFERENCES

1. D. Falanga, K. Kleber, S. Mintchevy, D. Floreanoy and D. Scaramuzza: The Foldable Drone: A Morphing Quadrotor that can Squeeze and Fly.
2. Piotr Kardasz, Jacek Doscocz, Mateusz Hejduk², PawelWiejkutand Hubert Zarzycki: Drones and Possibilities Using(<http://dx.doi.org/10.4172/2165-784X.1000233>)
3. Bas Vergouw, Huub Nagel, Geert Bondt and Bart CustersDrone Technology: Types, Payloads, Applications, Frequency Spectrum Issuesand Future Developments
4. Burchan Aydin, EmreSelvi, Jian Tao and Michael J. Starek Use of Fire-Extinguishing Balls for a Conceptual System of Drone-Assisted Wildfire Fighting
5. Dr. Ronald T. Wakeham & Dr. John C. Griffith, “Unmanned Aerial Systems in the Fire Service: Concepts and Issues.” JMER, June 2018, eISSN: 2536-8
6. Manuj C., Adarsh M Rao, Rahul S, Suhas C N, Vismay K G, “Design and Development of Semiautonomous fire Fighting Drone” Journal of Mechanical and Civil Engineering (IOSR-JMCE) June 2019, e-ISSN: 2394-3343.
7. Deruyck, M., Wyckmans, J., Martens, L., and Joseph, W. (2016), Emergency ad-hoc networks by using drone mounted base stations for a disaster scenario. IEEE 12th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp. 1-7
8. Galle, Jonas, and Verhelst, S. (2013). Design of a fast-responding start-up mechanism for bi-propellant fueled engine for miniature UAV applications. SAE technical paper series. Presented at the AeroTech Congress and Exhibition (SAE - 2013).
9. Emad E., M. Skriver, K. Husum, T. KjeldJensen (2017), A survey of OpenSource UAV flight controllers and flight simulators, Microprocessors and Microsystems, vol. 61, september 2017, pp. 11-20

APPENDIX

I. CODE

FLIGHT CONTROLLER:

```
#include <Wire.h> //Include the Wire.h library so we can communicate
with the gyro.
#include <EEPROM.h> //Include the EEPROM.h library so we can store information
onto the EEPROM

////////////////////////////////////
////////////////////////////////////
//PID gain and limit
settings
////////////////////////////////////
////////////////////////////////////

float pid_p_gain_roll = 1.3; //Gain setting for the roll P-
controller
float pid_i_gain_roll = 0; //Gain setting for the roll I-
controller
float pid_d_gain_roll = 8; //Gain setting for the roll D-
controller
int pid_max_roll = 400; //Maximum output of the PID-
controller (+/-)

float pid_p_gain_pitch = pid_p_gain_roll; //Gain setting for the pitch P-
controller.
float pid_i_gain_pitch = pid_i_gain_roll; //Gain setting for the pitch I-
controller.
float pid_d_gain_pitch = pid_d_gain_roll; //Gain setting for the pitch D-
controller.
int pid_max_pitch = pid_max_roll; //Maximum output of the PID-
controller (+/-)

float pid_p_gain_yaw = 3.0; //Gain setting for the pitch P-
controller. //4.0
float pid_i_gain_yaw = 0.02; //Gain setting for the pitch I-
controller. //0.02
float pid_d_gain_yaw = 0.0; //Gain setting for the pitch D-
controller.
int pid_max_yaw = 400; //Maximum output of the PID-
```

controller (+/-)

boolean auto_level = true; //Auto level on (true) or
off (false)

////////////////////////////////////
////////////////////////////////////

//Declaring global
variables

////////////////////////////////////
////////////////////////////////////

byte last_channel_1, last_channel_2, last_channel_3,
last_channel_4;

byte
eeprom_data[36];

byte highByte,
lowByte;

volatile int receiver_input_channel_1, receiver_input_channel_2,
receiver_input_channel_3, receiver_input_channel_4;

int counter_channel_1, counter_channel_2, counter_channel_3, counter_channel_4,
loop_counter;

int esc_1, esc_2, esc_3,
esc_4;

int throttle,
battery_voltage;

int cal_int, start,
gyro_address;

int
receiver_input[5];

int
temperature;

int acc_axis[4],
gyro_axis[4];

float roll_level_adjust,
pitch_level_adjust;

long acc_x, acc_y, acc_z,
acc_total_vector;

unsigned long timer_channel_1, timer_channel_2, timer_channel_3, timer_channel_4,
esc_timer, esc_loop_timer;

unsigned long timer_1, timer_2, timer_3, timer_4,
current_time;

unsigned long
loop_timer;

```

double gyro_pitch, gyro_roll,
gyro_yaw;
double
gyro_axis_cal[4];
float
pid_error_temp;
float pid_i_mem_roll, pid_roll_setpoint, gyro_roll_input, pid_output_roll,
pid_last_roll_d_error;
float pid_i_mem_pitch, pid_pitch_setpoint, gyro_pitch_input, pid_output_pitch,
pid_last_pitch_d_error;
float pid_i_mem_yaw, pid_yaw_setpoint, gyro_yaw_input, pid_output_yaw,
pid_last_yaw_d_error;
float angle_roll_acc, angle_pitch_acc, angle_pitch,
angle_roll;
boolean
gyro_angles_set;

////////////////////////////////////
////////////////////////////////////

//Setup
routine

////////////////////////////////////
////////////////////////////////////

void
setup(){
//Serial.begin(5760
0);
//Copy the EEPROM data for fast access
data.
for(start = 0; start <= 35; start++)eeprom_data[start] =
EEPROM.read(start);
start = 0; //Set start back to
zero.
gyro_address = eeprom_data[32]; //Store the gyro address in the
variable.

Wire.begin(); //Start the I2C as
master.

TWBR = 12; //Set the I2C clock speed to
400kHz.

//Arduino (Atmega) pins default to inputs, so they don't need to be explicitly
declared as inputs.
DDRD |= B11110000; //Configure digital poort 4, 5, 6 and 7

```

```

as output.
DDRB |= B00110000; //Configure digital poort 12 and 13
as output.

//Use the led on the Arduino for startup
indication.
digitalWrite(12,HIGH); //Turn on the
warning led.

//Check the EEPROM signature to make sure that the setup program is
executed.
while(eeprom_data[33] != 'J' || eeprom_data[34] != 'M' || eeprom_data[35] !=
'B')delay(10);

//The flight controller needs the MPU-6050 with gyro and
accelerometer
//If setup is completed without MPU-6050 stop the flight controller
program
if(eeprom_data[31] == 2 || eeprom_data[31] ==
3)delay(10);

set_gyro_registers(); //Set the specific gyro
registers.

for (cal_int = 0; cal_int < 1250 ; cal_int ++){ //Wait 5 seconds before
continuing.
PORTD |= B11110000; //Set digital poort 4, 5, 6 and
7 high.
delayMicroseconds(1000); //Wait
1000us.
PORTD &= B00001111; //Set digital poort 4, 5, 6
and 7 low.
delayMicroseconds(3000); //Wait
3000us.
}

//Let's take multiple gyro data samples so we can determine the average gyro offset
(calibration).
for (cal_int = 0; cal_int < 2000 ; cal_int ++){ //Take 2000 readings for
calibration.
if(cal_int % 15 == 0)digitalWrite(12, !digitalRead(12)); //Change the led status to
indicate calibration.
gyro_signalen(); //Read the gyro
output.
gyro_axis_cal[1] += gyro_axis[1]; //Ad roll
value to gyro_roll_cal.

```

```

gyro_axis_cal[2] += gyro_axis[2]; //Ad pitch
value to gyro_pitch_cal.
gyro_axis_cal[3] += gyro_axis[3]; //Ad yaw
value to gyro_yaw_cal.
//We don't want the esc's to be beeping annoyingly. So let's give
them a 1000us puls while calibrating the gyro.
PORTD |= B11110000; //Set digital
poort 4, 5, 6 and 7 high.
delayMicroseconds(1000);
//Wait 1000us.
PORTD &= B00001111; //Set digital
poort 4, 5, 6 and 7 low.
delay(3); //Wait 3 milliseconds
before the next loop.
}
//Now that we have 2000 measures, we need to devide by 2000
to get the average gyro offset.
gyro_axis_cal[1] /= 2000; //Divide the
roll total by 2000.
gyro_axis_cal[2] /= 2000; //Divide the
pitch total by 2000.
gyro_axis_cal[3] /= 2000; //Divide the
yaw total by 2000.

PCICR |= (1 << PCIE0); //Set PCIE0 to
enable PCMSK0 scan.
PCMSK0 |= (1 << PCINT0); //Set PCINT0 (digital input 8) to
trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT1); //Set PCINT1 (digital input 9)to
trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT2); //Set PCINT2 (digital input 10)to
trigger an interrupt on state change.
PCMSK0 |= (1 << PCINT3); //Set PCINT3 (digital input 11)to
trigger an interrupt on state change.

//Wait until the receiver is active and the throtle is
set to the lower position.
while(receiver_input_channel_3 < 990 ||
receiver_input_channel_3 > 1020 || receiver_input_channel_4 <
1400){
receiver_input_channel_3 = convert_receiver_channel(3);
//Convert the actual receiver signals for throttle to the standard
1000 - 2000us

```

```

receiver_input_channel_4 = convert_receiver_channel(4);
//Convert the actual receiver signals for yaw to the standard 1000 -
2000us
start ++; //While waiting increment
start whith every loop.
//We don't want the esc's to be beeping annoyingly. So let's give
them a 1000us puls while waiting for the receiver inputs.
PORTD |= B11110000; //Set digital
poort 4, 5, 6 and 7 high.
delayMicroseconds(1000);
//Wait 1000us.
PORTD &= B00001111; //Set digital
poort 4, 5, 6 and 7 low.
delay(3); //Wait 3 milliseconds
before the next loop.
if(start == 125){ //Every 125
loops (500ms).
digitalWrite(12, !digitalRead(12));
//Change the led status.
start = 0; //Start
again at 0.
}
}
start = 0; //Set start
back to 0.

//Load the battery voltage to the
battery_voltage variable.
//65 is the voltage
compensation for the diode.
//12.6V equals ~5V @
Analog 0.
//12.6V equals 1023
analogRead(0).
//1260 / 1023 =
1.2317.
//The variable battery_voltage holds 1050 if the
battery voltage is 10.5V.
battery_voltage = (analogRead(0)
+ 65) * 1.2317;

loop_timer = micros(); //Set the timer
for the next loop.

```

```

//When everything is done,
turn off the led.
digitalWrite(12,LOW); //Turn off
the warning led.
}

/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

//Main
program loop
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

void
loop(){

//65.5 = 1 deg/sec (check the datasheet of the MPU-
6050 for more information).
gyro_roll_input = (gyro_roll_input * 0.7) + ((gyro_roll / 65.5) *
0.3); //Gyro pid input is deg/sec.
gyro_pitch_input = (gyro_pitch_input * 0.7) + ((gyro_pitch / 65.5)
* 0.3); //Gyro pid input is deg/sec.
gyro_yaw_input = (gyro_yaw_input * 0.7) + ((gyro_yaw / 65.5) *
0.3); //Gyro pid input is deg/sec.

/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

//This is the added IMU code
from the videos:
//https://youtu.be/4BoI
E8YQwM8
//https://youtu.be/j-
kE0AMEWy4
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////

//Gyro angle
calculations
//0.0000611 = 1 /
(250Hz / 65.5)
angle_pitch += gyro_pitch * 0.0000611; //Calculate the traveled
pitch angle and add this to the angle_pitch variable.
angle_roll += gyro_roll * 0.0000611; //Calculate the traveled roll
angle and add this to the angle_roll variable.

//0.000001066 = 0.0000611 * (3.142(PI) / 180degr) The

```

```

Arduino sin function is in radians
angle_pitch -= angle_roll * sin(gyro_yaw * 0.000001066); //If the
IMU has yawed transfer the roll angle to the pitch angel.
angle_roll += angle_pitch * sin(gyro_yaw * 0.000001066); //If the
IMU has yawed transfer the pitch angle to the roll angel.

//Accelerometer angle
calculations
acc_total_vector =
sqrt((acc_x*acc_x)+(acc_y*acc_y)+(acc_z*acc_z)); //Calculate the
total accelerometer vector.

if(abs(acc_y) < acc_total_vector){ //Prevent the asin
function to produce a NaN
angle_pitch_acc = asin((float)acc_y/acc_total_vector)*
57.296; //Calculate the pitch angle.
}
if(abs(acc_x) < acc_total_vector){ //Prevent the asin
function to produce a NaN
angle_roll_acc = asin((float)acc_x/acc_total_vector)* -
57.296; //Calculate the roll angle.
}

//Place the MPU-6050 spirit level and note the values in the
following two lines for calibration.
angle_pitch_acc -= 0.0; //Accelerometer
calibration value for pitch.
angle_roll_acc -= 0.0; //Accelerometer
calibration value for roll.

angle_pitch = angle_pitch * 0.9996 + angle_pitch_acc * 0.0004;
//Correct the drift of the gyro pitch angle with the accelerometer
pitch angle.
angle_roll = angle_roll * 0.9996 + angle_roll_acc * 0.0004;
//Correct the drift of the gyro roll angle with the accelerometer roll
angle.

pitch_level_adjust = angle_pitch * 15; //Calculate
the pitch angle correction
roll_level_adjust = angle_roll * 15; //Calculate
the roll angle correction

if(!auto_level){ //If the quadcopter is not
in auto-level mode
pitch_level_adjust = 0; //Set the pitch
angle correction to zero.

```

```

roll_level_adjust = 0; //Set the roll
angle correction to zero.
}

//For starting the motors: throttle low
and yaw left (step 1).
if(receiver_input_channel_3 < 1050 &&
receiver_input_channel_4 < 1050)start = 1;
//When yaw stick is back in the center position
start the motors (step 2).
if(start == 1 && receiver_input_channel_3 < 1050 &&
receiver_input_channel_4 > 1450){
start =
2;

angle_pitch = angle_pitch_acc; //Set the gyro pitch angle equal to
the accelerometer pitch angle when the quadcopter is started.
angle_roll = angle_roll_acc; //Set the gyro roll angle equal to the
accelerometer roll angle when the quadcopter is started.
gyro_angles_set = true; //Set the
IMU started flag.

//Reset the PID controllers for a
bumpless start.
pid_i_mem_r
oll = 0;
pid_l
ast_r
oll_d
_erro
r = 0;
pid_
i_m
em_
pitc
h =
0;
pid_l
ast_pi
tch_d
_error
= 0;
pid_
i_m
em_

```

```

yaw
= 0;
pid_l
ast_y
aw_d
_erro
r = 0;
}
//Stopping
the
motors:
throttle
low and
yaw right.
if(start == 2 &&
receiver_input_channel_3
< 1050 &&
receiver_input_channel_4
> 1950)start = 0;

//The PID set
point in degrees
per second is
determined by
the roll receiver
input.
//In the case of deviding
by 3 the max roll rate is
aprox 164 degrees per
second ( (500-8)/3 =
164d/s ).
pid_
roll_
setp
oint
= 0;
//We need
a little
dead band
of 16us for
better
results.
if(receiver_input_c
hannel_1 >
1508)pid_roll_setp

```

```

oint =
receiver_input_channel_1 - 1508;
else
if(receiver_input_channel_1 <
1492)pid_roll_setpoint =
receiver_input_channel_1 - 1492;

pid_roll_setpoint -=
roll_level_adjust;
//Subtract the angle
correction from the
standardized receiver roll
input value.
pid_roll_setpoint /= 3.0;
//Divide the setpoint for
the PID roll controller by
3 to get angles in degrees.

//The PID set
point in degrees
per second is
determined by
the pitch receiver
input.
//In the case of deviding
by 3 the max pitch rate is
aprox 164 degrees per
second ( (500-8)/3 =
164d/s ).
pid_pitch_setpoint
= 0;
//We need
a little
dead band
of 16us for
better
results.
if(receiver_input_channel_2 >

```

```

1508)pid_pitch_set
point =
receiver_input_channel_2 - 1508;
else
if(receiver_input_channel_2 <
1492)pid_pitch_setpoint =
receiver_input_channel_2 - 1492;

pid_pitch_setpoint -=
pitch_level_adjust;
//Subtract the angle
correction from the
standardized receiver
pitch input value.
pid_pitch_setpoint /= 3.0;
//Divide the setpoint for
the PID pitch controller
by 3 to get angles in
degrees.

//The PID set
point in degrees
per second is
determined by
the yaw receiver
input.

//In the case of deviding
by 3 the max yaw rate is
aprox 164 degrees per
second ( (500-8)/3 =
164d/s ).

pid_yaw_setpoint =
0;

//We need
a little
dead band
of 16us for
better

```

```

results.
if(receiver_input
_channel_3 >
1050){ //Do not
yaw when
turning off the
motors.
if(receiver_input_ch
annel_4 >
1508)pid_yaw_setp
oint =
(receiver_input_cha
nnel_4 - 1508)/3.0;
else
if(receiver_input_channel
_4 <
1492)pid_yaw_setpoint =
(receiver_input_channel_
4 - 1492)/3.0;
}

calculate_pid();
//PID inputs are
known. So we
can calculate
the pid output.

//The
battery
voltage is
needed for
compensa
tion.
//A
compleme
ntary
filter is
used to
reduce
noise.
//0.09
853 =
0.08
*
1.231
7.

```

```

battery_voltage
=
battery_voltage
* 0.92 +
(analogRead(0)
+ 65) *
0.09853;

//Turn on
the led if
battery
voltage
is to low.
if(battery_voltag
e < 1000 &&
battery_voltage
>
600)digitalWrite
(12, HIGH);

throttle =
receiver_input_c
hannel_3; //We
need the throttle
signal as a base
signal.

if (start
==
2){ //Th
e
motors
are
started.

if (throttle > 1800)
throttle = 1800;
//We need some
room to keep full
control at full
throttle.

esc_1 = throttle -
pid_output_pitch +
pid_output_roll -
pid_output_yaw;
//Calculate the pulse for
esc 1 (front-right - CCW)

```

```

esc_2 = throttle +
pid_output_pitch +
pid_output_roll +
pid_output_yaw;
//Calculate the pulse for
esc 2 (rear-right - CW)
esc_3 = throttle +
pid_output_pitch -
pid_output_roll -
pid_output_yaw;
//Calculate the pulse for
esc 3 (rear-left - CCW)
esc_4 = throttle -
pid_output_pitch -
pid_output_roll +
pid_output_yaw;
//Calculate the pulse for
esc 4 (front-left - CW)

if
(battery_voltage
< 1240 &&
battery_voltage >
800){ //Is the
battery
connected?
esc_1 += esc_1 * ((1240 -
battery_voltage)/(float)35
00); //Compensate the esc-
1 pulse for voltage drop.
esc_2 += esc_2 * ((1240 -
battery_voltage)/(float)35
00); //Compensate the esc-
2 pulse for voltage drop.
esc_3 += esc_3 * ((1240 -
battery_voltage)/(float)35
00); //Compensate the esc-
3 pulse for voltage drop.
esc_4 += esc_4 * ((1240 -
battery_voltage)/(float)35
00); //Compensate the esc-
4 pulse for voltage drop.
}

if (esc_1 <
1100) esc_1

```

```
= 1100;
//Keep the
motors
running.
if (esc_2 <
1100) esc_2
= 1100;
//Keep the
motors
running.
if (esc_3 <
1100) esc_3
= 1100;
//Keep the
motors
running.
if (esc_4 <
1100) esc_4
= 1100;
//Keep the
motors
running.

if(esc_1 >
2000)esc_1 =
2000; //Limit
the esc-1
pulse to
2000us.
if(esc_2 >
2000)esc_2 =
2000; //Limit
the esc-2
pulse to
2000us.
if(esc_3 >
2000)esc_3 =
2000; //Limit
the esc-3
pulse to
2000us.
if(esc_4 >
2000)esc_4 =
2000; //Limit
the esc-4
```

```

pulse to
2000us.
}

e
l
s
e
{
esc_1 =
1000; //If
start is not 2
keep a
1000us pulse
for ess-1.
esc_2 =
1000; //If
start is not 2
keep a
1000us pulse
for ess-2.
esc_3 =
1000; //If
start is not 2
keep a
1000us pulse
for ess-3.
esc_4 =
1000; //If
start is not 2
keep a
1000us pulse
for ess-4.
}

//////////
//////////
//////////
//////////
//////////
////////
//Creating
the pulses
for the
ESC's is
explained in

```

this video:

//https:

//youtu

u.be/fq

EkVcq

xtU8

//////////

//////////

//////////

//////////

//////////

////

//!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!

//Because of the
angle calculation
the loop time is
getting very
important. If the
loop time is

//longer or shorter
than 4000us the
angle calculation is
off. If you modify
the code make sure

//that the loop
time is still
4000us and no
longer! More
information can
be found on

//th

e

Q&

A

pag

e:

//!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!

!!!!!!!!!!!!

```
if(micros() - loop_timer >
4050)digitalWrite(12,
HIGH); //Turn on the
LED if the loop time
exceeds 4050us.
```

```
//All the
information
for
controlling
the motor's
is available.
```

```
//The refresh
rate is 250Hz.
That means the
esc's need there
pulse every
4ms.
```

```
while(micros()
- loop_timer <
4000); //We
wait until
4000us are
passed.
```

```
loop_timer
= micros();
//Set the
timer for
the next
loop.
```

```
PORTD |=
B11110000;
//Set digital
outputs 4,5,6
and 7 high.
```

```
timer_channel_1 =
esc_1 + loop_timer;
//Calculate the time
of the falling edge of
the esc-1 pulse.
```

```
timer_channel_2 =
esc_2 + loop_timer;
//Calculate the time
of the falling edge of
```

```

the esc-2 pulse.
timer_channel_3 =
esc_3 + loop_timer;
//Calculate the time
of the falling edge of
the esc-3 pulse.
timer_channel_4 =
esc_4 + loop_timer;
//Calculate the time
of the falling edge of
the esc-4 pulse.

//There is always
1000us of spare
time. So let's do
something usefull
that is very time
consuming.

//Get the current
gyro and receiver
data and scale it to
degrees per second
for the pid
calculations.

gyr
o_s
ign
ale
n();

while(PORTD
>= 16){ //Stay
in this loop
until output
4,5,6 and 7 are
low.

esc_loop_t
imer =
micros();
//Read the
current
time.

if(timer_channel_1 <=
esc_loop_timer)PORTD
&= B11101111; //Set
digital output 4 to low if

```

```

the time is expired.
if(timer_channel_2 <=
esc_loop_timer)PORTD
&= B11011111; //Set
digital output 5 to low if
the time is expired.
if(timer_channel_3 <=
esc_loop_timer)PORTD
&= B10111111; //Set
digital output 6 to low if
the time is expired.
if(timer_channel_4 <=
esc_loop_timer)PORTD
&= B01111111; //Set
digital output 7 to low if
the time is expired.
}
}

////////////////////
////////////////////
////////////////////
////////////////////
////////////////////
//////

//This routine is called
every time input 8, 9, 10
or 11 changed state. This
is used to read the receiver
signals.
//More
information
about this
subroutine can
be found in
this video:
//https:
//youtu
.be/bE
Njl1K
Qbvo
////////////////////
////////////////////
////////////////////
////////////////////

```

```

////////////////////
/////
ISR
(PC
INT
0_v
ect)
{
curre
nt_ti
me =
micr
os();
//Channel
1=====
=====
=====
=====
==
if(PINB &
B00000001){
//Is input 8
high?
if(last_channel
_1 ==
0){ //Input 8
changed from 0
to 1.
last_channel_1
= 1;
//Remember
current input
state.
timer_1 =
current_time;
//Set timer_1
to
current_time.
}
}
else
if(last_channel_1 ==
1){ //Input 8 is not
high and changed

```

```

from 1 to 0.
last_channel_1
= 0;
//Remember
current input
state.
receiver_input[1] =
current_time -
timer_1; //Channel 1
is current_time -
timer_1.
}
//Channel
2=====
=====
=====

if(PINB &
B00000010
){ //Is input
9 high?
if(last_channel
_2 ==
0){ //Input 9
changed from 0
to 1.
last_channel_2
= 1;
//Remember
current input
state.
timer_2 =
current_time;
//Set timer_2
to
current_time.
}
}
else
if(last_channel_2 ==
1){ //Input 9 is not
high and changed
from 1 to 0.
last_channel_
2 = 0;

```

```

//Remember
current input
state.
receiver_input[2] =
current_time -
timer_2; //Channel 2
is current_time -
timer_2.
}
//Channel
3=====
=====
=====
===
if(PINB &
B00000100
){ //Is input
10 high?
if(last_channel
_3 ==
0){ //Input 10
changed from
0 to 1.
last_channel_
3 = 1;
//Remember
current input
state.
timer_3 =
current_time;
//Set timer_3
to
current_time.
}
}
else
if(last_channel_3 ==
1){ //Input 10 is not
high and changed
from 1 to 0.
last_channel_
3 = 0;
//Remember
current input

```

```

state.
receiver_input[3] =
current_time -
timer_3; //Channel 3
is current_time -
timer_3.

}
//Channel
4=====
=====
=====
===

if(PINB &
B00001000
){ //Is input
11 high?
if(last_channel
_4 ==
0){ //Input 11
changed from
0 to 1.
last_channel_
4 = 1;
//Remember
current input
state.
timer_4 =
current_time;
//Set timer_4
to
current_time.
}
}
else
if(last_channel_4 ==
1){ //Input 11 is not
high and changed
from 1 to 0.
last_channel_
4 = 0;
//Remember
current input
state.

```

```

receiver_input[4] =
current_time -
timer_4; //Channel 4
is current_time -
timer_4.
}
}

////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
//Subrou
tine for
reading
the gyro
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
void
gyro_
signal
en(){
//Rea
d the
MPU-
6050
if(eepr
om_da
ta[31]
== 1){
Wire.beginTransmis
sion(gyro_address);
//Start
communication with
the gyro.
Wire.write(0x3B);
//Start reading @
register 43h and auto
increment with every
read.
Wire.endTra

```

```

nsmission();
//End the
transmission
.
Wire.requestFrom(
gyro_address,14);
//Request 14 bytes
from the gyro.

receiver_input_channel_1
=
convert_receiver_channel(
1); //Convert the actual
receiver signals for pitch
to the standard 1000 -
2000us.
receiver_input_channel_2
=
convert_receiver_channel(
2); //Convert the actual
receiver signals for roll to
the standard 1000 -
2000us.
receiver_input_channel_3
=
convert_receiver_channel(
3); //Convert the actual
receiver signals for
throttle to the standard
1000 - 2000us.
receiver_input_channel_4
=
convert_receiver_channel(
4); //Convert the actual
receiver signals for yaw to
the standard 1000 -
2000us.

while(Wire.available() < 14); //Wait
until the 14 bytes
are received.
acc_axis[1] =
Wire.read()<<8|Wire.read(); //Add the low and
high byte to the acc_x

```

```

variable.
acc_axis[2] =
Wire.read()<<8|Wire.read(
d()); //Add the low and
high byte to the acc_y
variable.
acc_axis[3] =
Wire.read()<<8|Wire.read(
d()); //Add the low and
high byte to the acc_z
variable.
temperature =
Wire.read()<<8|Wire.read(
); //Add the low and high
byte to the temperature
variable.
gyro_axis[1] =
Wire.read()<<8|Wire.rea
d(); //Read high and
low part of the angular
data.
gyro_axis[2] =
Wire.read()<<8|Wire.re
ad(); //Read high and
low part of the angular
data.
gyro_axis[3] =
Wire.read()<<8|Wire.re
ad(); //Read high and
low part of the angular
data.
}

if(cal
_int
==
2000
){
gyro_axis[1] -=
gyro_axis_cal[1];
//Only compensate
after the
calibration.
gyro_axis[2] -=
gyro_axis_cal[2];

```

```

//Only compensate
after the
calibration.
gyro_axis[3] -=
gyro_axis_cal[3];
//Only compensate
after the
calibration.
}
gyro_roll =
gyro_axis[eprom_data[28
] & 0b00000011]; //Set
gyro_roll to the correct
axis that was stored in the
EEPROM.
if(eprom_data[28] &
0b10000000)gyro_roll *=
-1; //Invert gyro_roll if the
MSB of EEPROM bit 28
is set.
gyro_pitch =
gyro_axis[eprom_data[29
] & 0b00000011]; //Set
gyro_pitch to the correct
axis that was stored in the
EEPROM.
if(eprom_data[29] &
0b10000000)gyro_pitch
*= -1; //Invert gyro_pitch
if the MSB of EEPROM
bit 29 is set.
gyro_yaw =
gyro_axis[eprom_data[30
] & 0b00000011]; //Set
gyro_yaw to the correct
axis that was stored in the
EEPROM.
if(eprom_data[30] &
0b10000000)gyro_yaw *=
-1; //Invert gyro_yaw if
the MSB of EEPROM bit
30 is set.

acc_x =
acc_axis[eprom_data[29]

```

```

& 0b00000011]; //Set
acc_x to the correct axis
that was stored in the
EEPROM.
if(eeprom_data[29] &
0b10000000)acc_x *= -1;
//Invert acc_x if the MSB
of EEPROM bit 29 is set.
acc_y =
acc_axis[eeprom_data[28]
& 0b00000011]; //Set
acc_y to the correct axis
that was stored in the
EEPROM.
if(eeprom_data[28] &
0b10000000)acc_y *= -1;
//Invert acc_y if the MSB
of EEPROM bit 28 is set.
acc_z =
acc_axis[eeprom_data[30]
& 0b00000011]; //Set
acc_z to the correct axis
that was stored in the
EEPROM.
if(eeprom_data[30] &
0b10000000)acc_z *= -1;
//Invert acc_z if the MSB
of EEPROM bit 30 is set.
}

////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
///
//Subrouti
ne for
calculatin
g pid
outputs
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
///

```

```

//The PID
controllers are
explained in part 5
of the YMFC-3D
video session:
//https://
youtu.be
/JBvnB0
279-Q
////////////////////////////////
////////////////////////////////
////////////////////////////////
////////////////////////////////
///
void
calcul
ate_p
id(){
//Rol
l
calc
ulati
ons
pid_error_tem
p =
gyro_roll_inp
ut -
pid_roll_setpo
int;
pid_i_mem_r
oll +=
pid_i_gain_ro
ll *
pid_error_tem
p;
if(pid_i_mem_rol
l >
pid_max_roll)pid
_i_mem_roll =
pid_max_roll;
else
if(pid_i_mem_roll <
pid_max_roll * -
1)pid_i_mem_roll =
pid_max_roll * -1;

```

```

pid_output_roll =
pid_p_gain_roll *
pid_error_temp +
pid_i_mem_roll +
pid_d_gain_roll *
(pid_error_temp -
pid_last_roll_d_error);
if(pid_output_roll
>
pid_max_roll)pid
_output_roll =
pid_max_roll;
else
if(pid_output_roll <
pid_max_roll * -
1)pid_output_roll =
pid_max_roll * -1;

pid_last_r
oll_d_erro
r =
pid_error_
temp;

//Pitc
h
calcu
latio
ns
pid_error_tem
p =
gyro_pitch_inp
ut -
pid_pitch_setp
oint;
pid_i_mem_pit
ch +=
pid_i_gain_pit
ch *
pid_error_temp
;
if(pid_i_mem_pitch
>
pid_max_pitch)pid
_i_mem_pitch =
pid_max_pitch;

```

```

else
if(pid_i_mem_pitch <
pid_max_pitch * -
1)pid_i_mem_pitch =
pid_max_pitch * -1;

pid_output_pitch =
pid_p_gain_pitch *
pid_error_temp +
pid_i_mem_pitch +
pid_d_gain_pitch *
(pid_error_temp -
pid_last_pitch_d_error);
if(pid_output_pitch
>
pid_max_pitch)pid
_output_pitch =
pid_max_pitch;
else
if(pid_output_pitch <
pid_max_pitch * -
1)pid_output_pitch =
pid_max_pitch * -1;

pid_last_pi
tch_d_erro
r =
pid_error_t
emp;

//Ya
w
calc
ulati
ons
pid_error_tem
p =
gyro_yaw_inp
ut -
pid_yaw_setp
oint;
pid_i_mem_ya
w +=
pid_i_gain_ya
w *
pid_error_tem
p;

```

```

if(pid_i_mem_yaw
>
pid_max_yaw)pid
_i_mem_yaw =
pid_max_yaw;
else
if(pid_i_mem_yaw <
pid_max_yaw * -
1)pid_i_mem_yaw =
pid_max_yaw * -1;

pid_output_yaw =
pid_p_gain_yaw *
pid_error_temp +
pid_i_mem_yaw +
pid_d_gain_yaw *
(pid_error_temp -
pid_last_yaw_d_error);
if(pid_output_yaw
>
pid_max_yaw)pid
_output_yaw =
pid_max_yaw;
else
if(pid_output_yaw <
pid_max_yaw * -
1)pid_output_yaw =
pid_max_yaw * -1;

pid_last_y
aw_d_erro
r =
pid_error_
temp;
}

//This part converts the
actual receiver signals to a
standardized 1000 – 1500
– 2000 microsecond value.
//The
stored data
in the
EEPROM
is used.
int

```

```

convert_rec
eiver_chann
el(byte
function){
byte channel,
reverse; //First
we declare
some local
variables
int low,
center,
high,
actual;
int
diff
ere
nce
;

channel =
eeprom_data[function +
23] & 0b00000111; //What
channel corresponds with
the specific function
if(eeprom_data[function +
23] &
0b10000000)reverse = 1;
//Reverse channel when
most significant bit is set
else reverse = 0;
//If the most
significant is not
set there is no
reverse

actual =
receiver_input[channel];
//Read the actual receiver
value for the
corresponding function
low =
(eeprom_data[channel * 2
+ 15] << 8) |
eeprom_data[channel * 2
+ 14]; //Store the low
value for the specific

```

```

receiver input channel
center =
(eeprom_data[channel * 2
- 1] << 8) |
eeprom_data[channel * 2 -
2]; //Store the center value
for the specific receiver
input channel
high =
(eeprom_data[channel * 2
+ 7] << 8) |
eeprom_data[channel * 2
+ 6]; //Store the high value
for the specific receiver
input channel

if(actual <
center){ //The actual
receiver value is
lower than the
center value
if(actual < low)actual =
low; //Limit the lowest
value to the value that was
detected during setup
difference = ((long)(center
- actual) * (long)500) /
(center - low); //Calculate
and scale the actual value
to a 1000 - 2000us value
if(reverse == 1)return
1500 + difference; //If the
channel is reversed
else return 1500 -
difference; //If the channel
is not reversed
}
else if(actual >
center){ //The actual
receiver value is higher
than the center value
if(actual > high)actual =
high; //Limit the lowest
value to the value that was
detected during setup

```

```

difference = ((long)(actual
- center) * (long)500) /
(high - center); //Calculate
and scale the actual value
to a 1000 - 2000us value
if(reverse == 1)return
1500 - difference; //If the
channel is reversed
else return 1500 +
difference; //If the channel
is not reversed
}
else return
1500;
}

void
set_gyro_registers(){
//Setup the MPU-
6050
if(eeprom_data[31] ==
1){
Wire.beginTransaction(g
yro_address); //Start
communication with the
address found during
search.
Wire.write(0x6B); //We
want to write to the
PWR_MGMT_1 register
(6B hex)
Wire.write(0x00); //Set the
register bits as 00000000
to activate the gyro
Wire.endTransmission();
//End the transmission
with the gyro.

Wire.beginTransaction(g
yro_address); //Start
communication with the
address found during
search.
Wire.write(0x1B); //We
want to write to the

```

GYRO_CONFIG register
(1B hex)

Wire.write(0x08); //Set the
register bits as 00001000
(500dps full scale)

Wire.endTransmission();
//End the transmission
with the gyro

Wire.beginTransmission(g
yro_address); //Start
communication with the
address found during
search.

Wire.write(0x1C); //We
want to write to the
ACCEL_CONFIG register
(1A hex)

Wire.write(0x10); //Set the
register bits as 00010000
(+/- 8g full scale range)

Wire.endTransmission();
//End the transmission
with the gyro

//Let's perform a random
register check to see if the
values are written correct

Wire.beginTransmission(g
yro_address); //Start
communication with the
address found during
search

Wire.write(0x1B); //Start
reading @ register 0x1B

Wire.endTransmission();
//End the transmission

Wire.requestFrom(gyro_a
ddress, 1); //Request 1
bytes from the gyro

while(Wire.available() <
1); //Wait until the 6 bytes
are received

if(Wire.read() !=
0x08){ //Check if the

```

value is 0x08
digitalWrite(12,HIGH);
//Turn on the warning led
while(1)delay(10); //Stay
in this loop for ever
}

Wire.beginTransmission(g
yro_address); //Start
communication with the
address found during
search
Wire.write(0x1A); //We
want to write to the
CONFIG register (1A hex)
Wire.write(0x03); //Set the
register bits as 00000011
(Set Digital Low Pass
Filter to ~43Hz)
Wire.endTransmission();
//End the transmission
with the gyro
}
}

```

Fire extinguisher:

```

#include<ESP8266WiFi.h>
#include<ESP8266WebServer.h>
#include<SPI.h>
#include<Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>
#include<Adafruit_GFX.h>
#include<Adafruit_SSD1306.h>
#include <Servo.h>
#include<DHTesp.h>

#define SOUND_VELOCITY 0.034
#define CM_TO_INCH 0.393701
#define SEALEVELPRESSURE_HPA (1013.25)

//-----

ESP8266WebServer server(80);
Adafruit_SSD1306 display(128,64, &Wire,-1);

```

```

Adafruit_BMP280 bmp;
const char ssid[]="charan_cd";
const char password[]="123456789";
Servo myservo1;
DHTesp dht;

const int temp=0,flame=14,trig=12 ,echo=13,BUZZ=2;
//0=d3,14=d5,12=d6,13=d7,2=d4
const int ledPin = 16;
int flame_val=0;

long duration;
float distanceCm;
float distanceInch;
float humidity ;
float temperature;
float pres = bmp.readPressure()/100;
float alti = bmp.readAltitude(SEALEVELPRESSURE_HPA);
int flag1 = 0;
String flam,ball,obj;

//-----
void setup()
{
int i;

//buzzer is connected to D8 pin of nodemcu
pinMode(BUZZ,OUTPUT);
pinMode(flame,INPUT);
pinMode(trig,OUTPUT);
pinMode(echo,INPUT);
dht.setup(0, DHTesp::DHT22); // Connect DHT sensor to GPIO 5

for(i=0;i<2;i++)
{
digitalWrite(BUZZ,HIGH);
delay(50);
digitalWrite(BUZZ,LOW);
delay(50);
}

Serial.begin(9600);

//connect to your local wi-fi network
WiFi.begin(ssid, password);

//check wi-fi is connected to wi-fi network

```

```

while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected..!");
Serial.print("Got IP: "); Serial.println(WiFi.localIP());

server.on("/page",page);

server.begin();

Serial.println("HTTP server started");

myservo1.attach(15);//connect a servo to D8 pin of nodemcu
myservo1.write(180);

//connect the SCK and SDA pins of OLED to D1 and D2 pins of nodemcu respectively
Wire.begin();
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
if (!bmp.begin(0x76)) {
  // connection error or device address wrong!
  Serial.println(F("Could not find a valid BMP280 sensor, check wiring!"));
  display.setCursor(34, 23);
  display.print("Connection");
  display.setCursor(49, 33);
  display.print("Error");
  display.display(); // update the display
  while (1) // stay here
    delay(1000);
}

display.clearDisplay();

display.setTextSize(3);
display.setTextColor(WHITE);
display.setCursor (2,25);
display.println("WELCOME");
display.display();
delay(1000);
display.clearDisplay();

display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor (46,0);
display.println("FIRE");
display.setCursor(20,25);
display.println("EXTIGUISH");
display.setCursor (17,45);

```

```

display.println(" SYSTEM ");
display.display();
delay (1000);
display.clearDisplay();

}
//-----

void loop()
{
  server.handleClient();
  delay(1000);
  humidity = dht.getHumidity();
  temperature = dht.getTemperature();
  pres = bmp.readPressure()/100;
  alti = bmp.readAltitude(SEALEVELPRESSURE_HPA)-450.00;

  distance();

  flame_val=digitalRead(flame);

  if(flag1==1)
  {
    ball="DEPLOYED";
  }
  else
  {
    ball="READY";
  }

  if(flame_val==0||temperature>70)
  {
    flam="HIGH";
    digitalWrite(BUZZ, HIGH);
    delay(100);
    digitalWrite(BUZZ, LOW);
    delay(100);
  }
  else
  {
    flam="REDUCING";
  }
  display.clearDisplay();
  display.setTextSize(1);

```

```

display.setTextColor(WHITE);
display.setCursor (0,0);
display.println("TEMPERATURE:");
display.print(temperature);
display.setCursor (0,17);
display.println("FLAME STATUS:");
display.print(flam);
display.setCursor (0,38);
display.println("EXTINGUISHER:");
display.print(ball);
display.display();
delay(1000);

}
//-----
void distance(){
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    // Sets the trigPin on HIGH state for 10 micro seconds
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);

    // Reads the echoPin, returns the sound wave travel time in microseconds
    duration = pulseIn(echo, HIGH);

    // Calculate the distance
    distanceCm = duration * SOUND_VELOCITY/2;

    // Convert to inches
    distanceInch = distanceCm * CM_TO_INCH;

    if(distanceInch<25)
    {
        obj="BLOCKED";
        digitalWrite(BUZZ,HIGH);
        delay(100);
        digitalWrite(BUZZ,LOW );
        delay(100);
    }
    else
    {
        obj="CLEAR";
    }
}
//-----

void page()
{

```

```

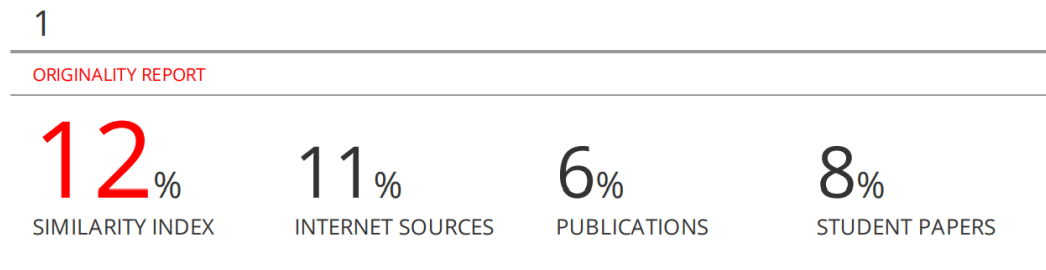
String mycode="<p><font size=\"4\" color=\"#80ced6\">"+String("TEMPERATURE:
")+"/font><mark>"+ float(temperature)+"/mark><br><font size=\"4\"
color=\"#80ced6\">"+String("HUMIDITY: ")+"</font><mark>"+
float(humidity)+"%</mark><br><font size=\"4\" color=\"#80ced6\">"+String("ALTITUDE:
")+"/font><mark>"+ float(alti)+"M</mark><br><font size=\"4\"
color=\"#80ced6\">"+String("PRESSURE: ")+"</font><mark>"+
float(pres)+"/mark><br><font size=\"4\" color=\"#80ced6\">"+String("PATHWAY:
")+"/font><mark>"+ String(obj)+"/mark><br><font size=\"4\"
color=\"#80ced6\">"+String("FIRE STATUS: ")+"</font><mark>"+
String(flam)+"/mark><br><font size=\"4\" color=\"#80ced6\">"+String("EXTINGUISHER:
")+"/font><mark>"+String(ball)+"/mark><br></p>";
mycode="<html><head><title>fire extinguisher</title></head><body><body
bgcolor=\"#00008b\"><font size=\"7\"
color=\"#ADD8E6\"><center><u><b><h1>STATUS</h1></b></u></font><b>"+mycode+"<fo
rm><button type=\"submit\" name=\"state\" value=\"1\">DROP</button><button
type=\"submit\" name=\"state\"
value=\"0\">RELOAD</button></form></center></b><meta http-equiv=\"refresh\"
content=\"2\"></body></html>";
server.send(200,"text/html",mycode);

if(server.arg("state")==="1")
{
myservo1.write(-180);
flag1=1;
}
if(server.arg("state")==="0")
{
myservo1.write(180);
flag1=0;
}
}

//-----

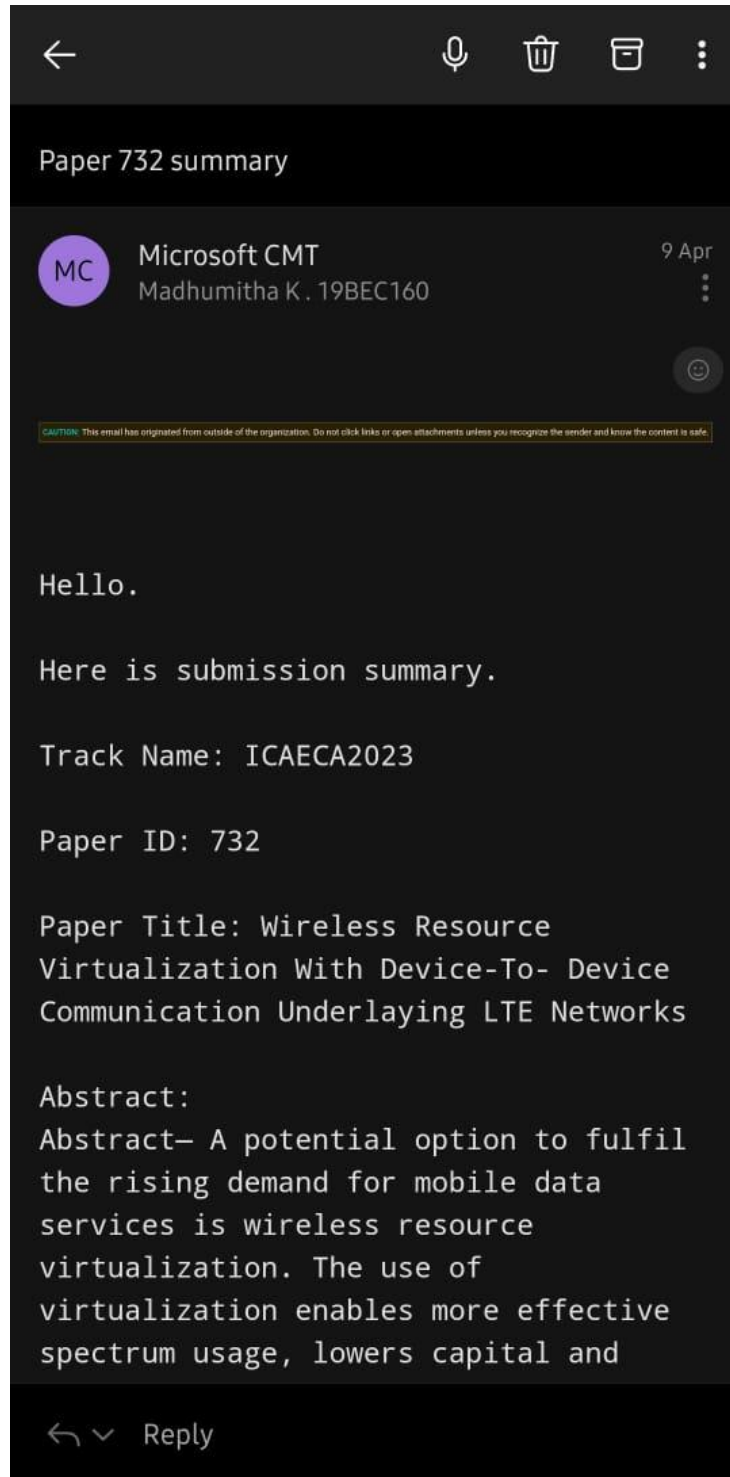
```

II. PLAGIARISM CHECKER



III. PAPER SUBMISSION

Phase 1 Paper



Phase 2 Paper

